Programming Methodology Lab2
Modern C++ & STL 1

Lab2

Fall 2025

Hyunwoong Bae (배현웅)
hwbae0326@snu.ac.kr

Machine Intelligence & Data science LAB

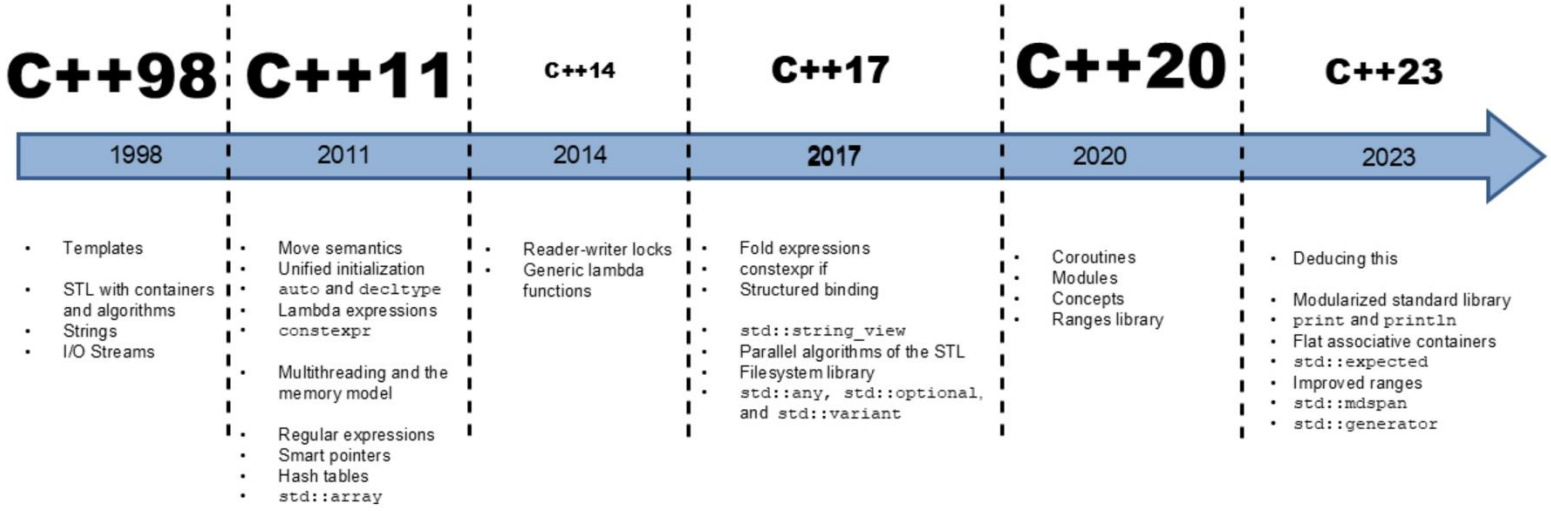
Modern C++

- Modern C++ (generally) refers to C++11 or later

C++26: The Next C++ Standard

August 19, 2024 / 0 Comments / in C++26 / by Rainer Grimm

C++26 will be the next C++ standard after C++23. This new standard significantly improves C++ and is probably similar game-changing like C++98, C++11, or C++20.



Outline

- 1. New feature in modern C++
  - Automatic Type Deduction (auto, decltype)
- 2. Tuple
- 3. STL in modern C++ : Introduction
- 4. Array
  - 1. Container
  - 2. Iterator
  - 3. Algorithm

Outline

- 1. New feature in modern C++
  - Automatic Type Deduction (auto, decltype)
- 2. Tuple
- 3. STL in modern C++ : Introduction
- 4. Array
  - 1. Container
  - 2. Iterator
  - 3. Algorithm

1. New Feature in Modern C++
1.1. Automatic Type Deduction – 'auto'

- In C++, you must specify the type of an object when you declare it.
  - Static programming language
  - Ex) Java, C#...

C++
```
void main(){
    int a = 1;
    char c = 'a';
    double d = 0.1;
}
```

- In dynamic programming language,
  - Type of variable is automatically decided in compile time.
  - Ex) Python, JavaScript ...

Python
```
x = 34 - 23
y = "Hello"
z = 3.45
```

Python
```
x = 34 - 23
print(x)
x = "Hello"
print(x)
```

1. New Feature in Modern C++
1.1. Automatic Type Deduction – 'auto'

- A keyword 'auto'
  - Compiler infers the variable's type.
  - Works with the return value from function.

```
void main(){
    auto d = 5.0;    <- double
    auto i = 1+2;    <- int
}
```

```
int add(int x, int y){
    return x+y;
}

void main(){
    auto sum = add(5,6);
}
```

Machine Intelligence & Data science LAB

1. New Feature in Modern C++
1.1. Automatic Type Deduction – 'auto'

- Note:
  - 'Auto' cannot be used without initialization
  - Also, cannot be used as a parameters of functions (before c++20)

C++20 allows `auto` as function parameter type

This code is valid using C++20:

```
int function(auto data) {
   // do something, there is no constraint on data
}
```


https://stackoverflow.com/questions/29944985/is-there-a-way-to-pass-auto-as-an-argument-in-c
https://mangu.tistory.com/201

1. New Feature in Modern C++
1.1. Automatic Type Deduction – 'auto'

- Why 'auto'?
  - When you use template or more advanced feature, defining type of return variable is difficult.

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> vec;
    vec.push_back(1);
    vec.push_back(3);
    vec.push_back(5);
    vec.push_back(7);

    cout << "Without auto keyword" << endl;
    for (vector<int>::iterator it = vec.begin(); it != vec.end(); it++)
    {
        cout << *it << " ";
    }
    cout << endl;
    cout << "With auto keyword" << endl;
    for (auto it = vec.begin(); it != vec.end(); it++)
    {
        cout << *it << " ";
    }
    cout << endl;
    return 0;
}
```

```
without auto keyword
1 3 5 7
with auto keyword
1 3 5 7
```

1. New Feature in Modern C++
1.2. Automatic Type Deduction – 'decltype'

- A keyword 'decltype'
  - Inspects the declared type of an entity
- Syntax
  - decltype(entity)

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    auto a = 2;
    decltype(a) b = 3;
    decltype(a + b) c = a + b;
    auto d = sqrt(a * a + b * b);
    cout << d << endl;
    return 0;
}
```

<- int
<- int
<- int
<- double

1. New Feature in Modern C++

1.3. Exercise # 1

```cpp
int main(void)
{
    // # 1. Assign the value 2 to the variable a

    cout << "1: " << a << endl;
    f(a);
    // # 2. Use the decltype of a to assign the value 3 to the variable b.

    cout << "2: " << b << endl;
    f(b);
    // # 3. Assign the results of (a+b) to the variable c using decltype

    cout << "3: " << c << endl;
    f(c);
    // # 4. Assign the results of sqrt of (a*a + b*b) to the variable d using decltype

    cout << "4: " << d << endl;
    f(d);

    return 0;
}
```

1. New Feature in Modern C++

1.3. Exercise # 1

```cpp
#include <iostream>
#include <cmath>
using namespace std;

void f(int i)
{
    cout << "integer type!" << endl;
}

void f(double d)
{
    cout << "double type!" << endl;
}

int main(void)
{
    // #1. type of a
    auto a = 2;
    cout << "1: " << a << endl;
    f(a);
    // #2. type of b
    decltype(a) b = 3;
    cout << "2: " << b << endl;
    f(b);
    // #3. type of c
    decltype(a + b) c = a + b;
    cout << "3: " << c << endl;
    f(c);
    // #4. type of d
    auto d = sqrt(a * a + b * b);
    cout << "4: " << d << endl;
    f(d);

    return 0;
}
```

```
1: 2
integer type!
2: 3
integer type!
3: 5
integer type!
4: 3.60555
double type!
```

Machine Intelligence & Data science LAB

Outline

- 1. New feature in modern C++
  - Automatic Type Deduction (auto, decltype)
- 2. Tuple
- 3. STL in modern C++ : Introduction
- 4. Array
  - 1. Container
  - 2. Iterator
  - 3. Algorithm

2. Tuple
2.1. What is the Tuple?

- What is the Tuple?
  - An **object** that can hold a collection of elements
  - Each elements can be a **different** type
  - Tuple object can be used as **input** and **output** of function

- Syntax
  - std::tuple<*type1*, *type2*, ...> NAME;

- Assigning values
  - NAME = std::tuple<type1, type2, ...> (val1, val2, ...);

2. Tuple
2.2. When to use?

- When to use tuple?
  - W/o tuple, we can return only a single variable

```
int add(int x, int y)
{
    return x + y;
}
```

  - How to define a function that returns multiple variables?
    - Ex) a function with return x//y and x%y

```
int division(int x, int y)
{
    //return quotient and remainder
}
```

2. Tuple
2.2. When to use?

- Method 1) return with array and vector
  - Cannot return variables with different d-type
- Method 2) reference
  - Too many parameters
- Method 3) use class or struct
  - Define unnecessary class

```
vector<int> division(int x, int y){
    vector<int> a;
    a.push_back(x/y);
    a.push_back(x-a[0]*y);
    return a;
}
```

Method 1

```
void division(int x, int y, int& quotient, int&
remainder){
    quotient = x/y;
    remainder = x-quotient*y;
}
```

Method 2

```
class Divide
{
public:
    Divide(){}
    Divide(int q, int r){quotient = q; remainder = r;}
    ~Divide(){}
    int quotient, remainder;
};

Divide division(int x, int y){
    return Divide(x/y, x-x/y*y);
}
```

Method 3

Machine Intelligence & Data science LAB

2. Tuple
2.2. Exercise #2

- Method 4) Using tuple

```cpp
#include <iostream>
#include <tuple>
using namespace std;

tuple<int, int> division(int a, int b)
{
    return make_tuple(a / b, a % b);
}

int main()
{
    auto result = division(7, 3);
    cout << "quotient: " << get<0>(result) << endl;
    cout << "remainder: " << get<1>(result) << endl;

    return 0;
}
```

```
quotient: 2
remainder: 1
```

2. Tuple
2.3. Operations

- Useful operations in tuple
  - Declaration
    - std::tuple<*type1*, *type2*, ...> NAME;
  - Assign a tuple with values
    - make_tuple(*val1*, *val2*, ...)
  - Accessing to value
    - get<index>(NAME)

2. Tuple
2.3. Operations

- Useful operations in tuple
  - Return a number of elements in tuple
    - tuple_size<decltype(NAME)>::value
  - Concatenate tuples
    - tuple_cat(*tuple1*, *tuple2*, …)

2. Tuple
2.4. Exercise #3

```cpp
#include <iostream>
#include <tuple>
#include <string>
using namespace std;

void print_tuple(tuple<string, int, int> t)
{
    cout << get<0>(t) << " " << get<1>(t) << " " << get<2>(t) << endl;
}

int main()
{
    // initialization of tuple
    tuple<string, int, int> tup1 = tuple<string, int, int>("josh", 1, 27);
    tuple<string, int, int> tup2 = make_tuple("josh", 4, 27);
    tuple<string, int, int> tup3("Nick", 8, 11);

    // print tuple with get function using tup1
    cout << "before changes: ";
    print_tuple(tup1);

    // change values of tuple with get() function
    cout << "after changes: ";
    get<0>(tup1) = "Joshua";
    print_tuple(tup1);

    // concat tupe (tup1,tup2)
    auto tup4 = tuple_cat(tup1, tup2);
    cout << "size of tup4: " << tuple_size<decltype(tup4)>::value << endl;
    cout << get<0>(tup4) << " " << get<1>(tup4) << " " << get<2>(tup4) << " ";
    cout << get<3>(tup4) << " " << get<4>(tup4) << " " << get<5>(tup4) << endl;

    return 0;
}
```

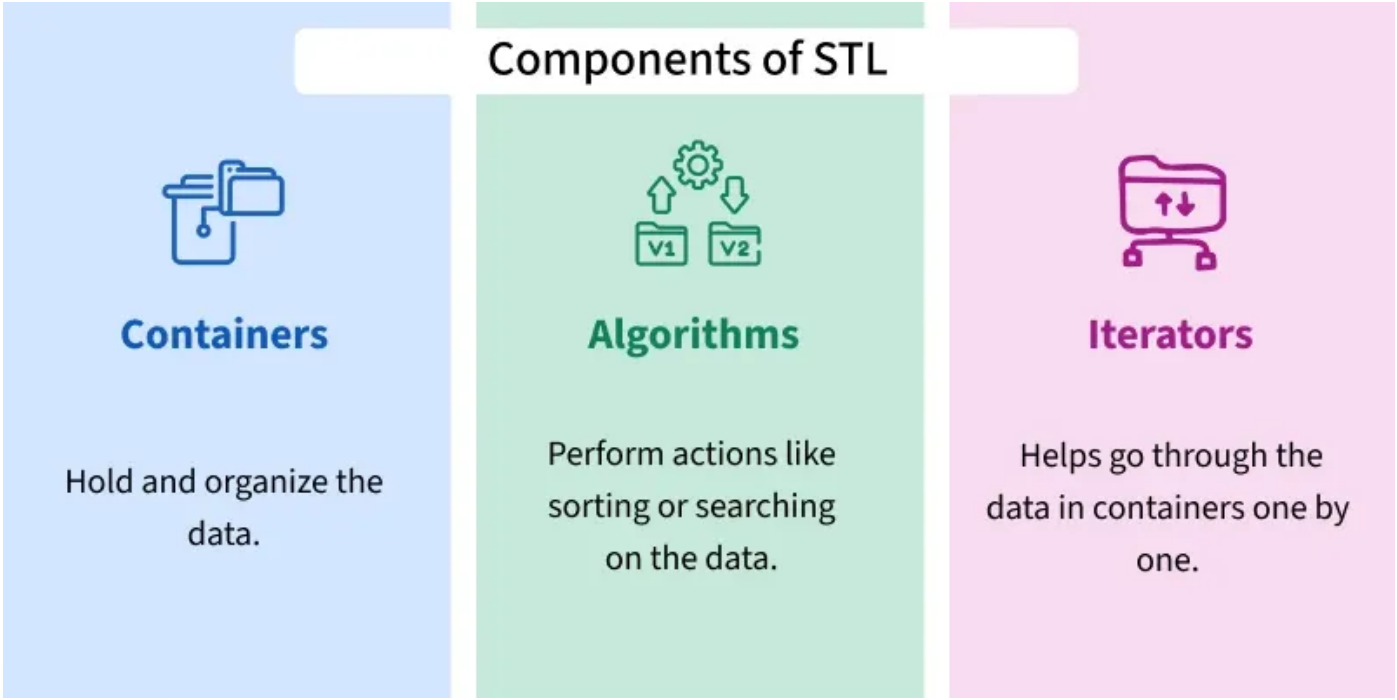
```
before changes: josh 1 27
after changes: Joshua 1 27
size of tup4: 6
Joshua 1 27 josh 4 27
```

Outline

- 1. New feature in modern C++
  - Automatic Type Deduction (auto, decltype)
- 2. Tuple
- 3. STL in modern C++ : Introduction
- 4. Array
  - 1. Container
  - 2. Iterator
  - 3. Algorithm

3. STL in modern C++
3.1. Introduction to STL

- The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc



https://www.geeksforgeeks.org/cpp/the-c-standard-template-library-stl/

3. STL in modern C++
3.1. Introduction to STL

- **Containers:**
  - Object which create and store (data & object)
  - **Sequence Containers**:
    - The data structures which can be accessed in a **sequential manner**.
    - Ex) array, vector, list, deque, forward_list
    - Elements of sequence containers can be accessed by index.
  - **Associative container**:
    - Ex) set, multiset, map, multimap
  - **Container adaptors**:
    - Ex) stack, queue, priority_queue

We will cover on the next time

Machine Intelligence & Data science LAB

3. STL in modern C++
3.1. Introduction to STL

- **Iterators**
  - Used as a pointer to point at the memory address of **STL containers**
    - Ex) begin(), end(), next(), prev(), advance(), ...

- **Algorithms**
  - Functions for a variety of purposes that operate on range of elements. Ex) Sorting, Searching, ...
  - The algorithm can be accessed with the help of **Iterator** only

Outline

- 1. New feature in modern C++
  - Automatic Type Deduction (auto, decltype)
- 2. Tuple
- 3. STL in modern C++ : Introduction
- 4. Array
  - 1. Container
  - 2. Iterator
  - 3. Algorithm

4. Array
4.1. Container

- What is the Array?
  - A container with fixed storage size.
- The advantages of STL array class over C-style array
  - Array class knows its own size
  - Light-weight, reliable, and efficient

- Syntax
  - array<data_type, size> name = {value1, value2, ..., valueN};

4. Array
4.1. Container – Member Functions

https://en.cppreference.com/w/cpp/container/array

- Member functions of Array class

Element access	
`at` (C++11)	access specified element with bounds checking (public member function)
`operator[]` (C++11)	access specified element (public member function)
`front` (C++11)	access the first element (public member function)
`back` (C++11)	access the last element (public member function)
`data` (C++11)	direct access to the underlying array (public member function)
Iterators	
`begin` `cbegin` (C++11)	returns an iterator to the beginning (public member function)
`end` `cend` (C++11)	returns an iterator to the end (public member function)
`rbegin` `crbegin` (C++11)	returns a reverse iterator to the beginning (public member function)
`rend` `crend` (C++11)	returns a reverse iterator to the end (public member function)
Capacity	
`empty` (C++11)	checks whether the container is empty (public member function)
`size` (C++11)	returns the number of elements (public member function)
`max_size` (C++11)	returns the maximum possible number of elements (public member function)
Operations	
`fill` (C++11)	fill the container with specified value (public member function)
`swap` (C++11)	swaps the contents (public member function)

Machine Intelligence & Data science

4. Array
4.1. Container – Exercise # 4

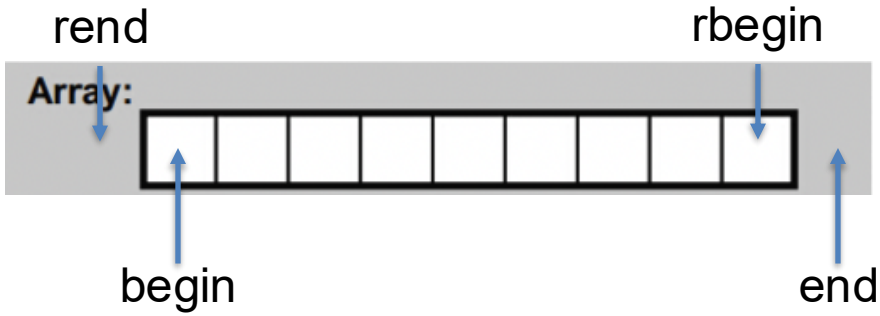
```cpp
#include <iostream>
#include <array> // for array, at()
#include <tuple> // for get()
#include <string>
using namespace std;

void print_array(array<int, 3> arr)
{
    for (int i = 0; i < arr.size(); i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main()
{
    // Initialization of array
    array<int, 3> arr1 = {4, 2, 3};

    cout << "The array elements are (using at(),[], get<>()) : ";
    cout << arr1.at(0) << " " << arr1[1] << " " << get<2>(arr1) << " " << endl;

    cout << "The array elements are (using a custom function) : ";
    print_array(arr1);

    cout << "First element of array is : " << arr1.front() << endl;
    cout << "Last element of array is : " << arr1.back() << endl;
    cout << "The size of array is : " << arr1.size() << endl;

    arr1.fill(0);
    cout << "The array elements after fill() are : ";
    print_array(arr1);

    return 0;
}
```

```
The array elements are (using at(), [], get<>()) : 4 2 3
The array elements are (using a custom function) : 4 2 3
First element is : 4
Last element is : 3
The size of array is : 3
The array elements are (after fill) : 0 0 0
```

Machine Intelligence & Data science LAB

4. Array
4.2. Iterator

- What is the Iterator?
  - It is used to point at the **memory addresses** of STL containers



  - Value in iterator can be accessed by * operator

Iterators	
`begin` `cbegin` (C++11)	returns an iterator to the beginning (public member function)
`end` `cend` (C++11)	returns an iterator to the end (public member function)
`rbegin` `crbegin` (C++11)	returns a reverse iterator to the beginning (public member function)
`rend` `crend` (C++11)	returns a reverse iterator to the end (public member function)

4. Array
4.2. Iterator – Practice

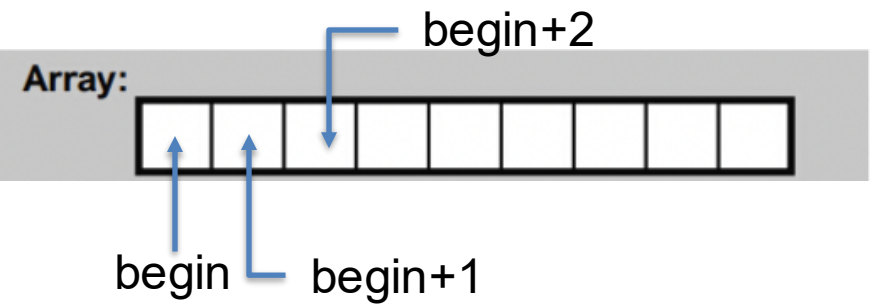
```cpp
#include <iostream>
#include <array>
using namespace std;

int main()
{
    array<int, 5> arr = {3, 2, 3, 4, 5};

    cout << arr.begin() << endl
         << arr.begin() + 1 << endl
         << arr.end() - 1 << endl
         << arr.end() << endl;
    cout << "Length : " << arr.end() - arr.begin() << endl;

    cout << *arr.begin() << endl
         << *(arr.begin() + 1) << endl
         << *(arr.end() - 1) << endl
         << *(arr.end()) << endl;
    return 0;
}
```

```
0x16d3ef130
0x16d3ef134
0x16d3ef140
0x16d3ef144
Length : 5
3
2
5
1
```

Machine Intelligence & Data science LAB

4. Array
4.2. Iterator – Exercise #5

- Recall the exercise #4.




```
0x400b3f
3 2 3 4 5
3 2 3 4 5
3 2 3 4 5
```

```cpp
#include <iostream>
#include <array>
using namespace std;
int main()
{
    array<int, 5> arr = {3, 2, 3, 4, 5};

    // Declaring iterator to a array
    array<int, 5>::iterator itr;
    cout << itr << endl;

    for (itr = arr.begin(); itr < arr.end(); itr++)
    {
        cout << *itr << " ";
    }
    cout << endl;

    // Use auto with iterator
    for (auto itr_ = arr.begin(); itr_ < arr.end(); itr_++)
    {
        cout << *itr_ << " ";
    }
    cout << endl;

    for (auto elem : arr)
    {
        cout << elem << " ";
    }
    cout << endl;

    return 0;
}
```

Machine Intelligence & Data science LAB

4. Array
4.3. Algorithm – Exercise #6

- **sort(first_iterator, last_iterator)**
  - To sort the given array.
- **reverse(first_iterator, last_iterator)**
  - To reverse a array.
- **find(first_iterator, last_iterator, value)**
  - To find an iterator with given value.

4. Array
4.3. Algorithm – Exercise #6

```cpp
int main()
{
    array<int, 4> arr = {4, 2, 3, 2};

    //implement code: sort the arr
    sort(arr.begin(), arr.end());
    cout << "The array elements are (using sort) :";
    print_array(arr);


    // implement code : find index of the value 4
    // task : print the result of find function and retrieve index from it.
    cout << "find an iterator with value 4 : ";

    // implement code : print the result of find function
    auto it = find(arr.begin(), arr.end(), 4);
    cout << it << endl;
    // implement code : retrieve index from it.
    auto index = it-arr.begin();
    // cout << arr.begin() << "," << it << "," << it-arr.begin() << endl;
    cout << "The 4 value is in "<< index << "th index " << endl;
```

```cpp
    // implement code : reverse the order of arr
    reverse(arr.begin(), arr.end());

    cout << "The array elements are (using reverse) :";
    print_array(arr);

    return 0;
}
```

```
The array elements are (using sort) :2 2 3 4
find an iterator with value 4 : 0x7fff49d6627c
The 4 value is in 3th index
The array elements are (using reverse) :4 3 2 2
```

Machine Intelligence & Data science LA

5. Assignment - Sieve of Eratosthenes

- Algorithm to find prime number

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

5. Assignment - Sieve of Eratosthenes

- You are given an integer number 2<n<300
- Write a c++ program that
  - [Initialize Part]
    - Initialize an array 'arr' with element {1, 2, ..., n}
    - make the enough size of array
  - [Algorithm Part]
    - Change the composite (non-prime) numbers to 0 in array.
      - 1 is not a prime number
      - You are not allowed to use [] or get operator in this part. Be familiar with iterator!
- Due date : 10 / 20 (Mon) 3:00 pm

Machine Intelligence & Data science LAB

5. Assignment
- Skeleton and Example Output

```cpp
#include <iostream>
#include <array>
#include <cmath>
using namespace std;
int main()
{
    int n;
    cout << "size of array: " << endl;
    cin >> n;

    // Initialization
    array<int, 500> arr;
    arr.fill(0);

    // Algorithm
    // int check;

    // Output (Do not modify)
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

```
30
0 2 3 0 5 0 7 0 0 0 11 0 13 0 0 0 17 0 19 0 0 0 23 0 0 0 0 0 29 0
```